$ rails new rubipticos

# TFI

# Ruby On Rails

## Rubípticos



**Cátedra de TTPS Opción Ruby**

Cursada 2026



# antes_de_empezar

# Antes de empezar

RUBY

## 3.3 o más

```
$ ruby -v
```

Instalarlo con un gestor de versiones: rbenv, asdf o mise.

RAILS

## 8.1

```
$ gem install rails
$ rails -v
```

La versión estable más reciente.

GIT

## GitHub

```
$ git --version
```

Un repositorio del grupo, con commits de todxs.

Guía: guides.rubyonrails.org/install_ruby_on_rails.html



# rails_new

# Crear el proyecto

EN LA CONSOLA

```
$ rails new rubipticos --css=bootstrap
$ cd rubipticos
$ bin/rails g authentication
$ bin/rails db:migrate
$ bin/dev
# abrir http://localhost:3000
```

QUÉ TRAE RAILS 8

**SQLite**
La base de datos, lista para usar.

**Hotwire**
Turbo y Stimulus para la interactividad.

**Autenticación**
Se agrega después, con el generador.

--css=bootstrap necesita Node.js y Yarn instalados. Sin Node: --css=tailwind, o Bootstrap desde su CDN en el layout (app/views/layouts/application.html.erb) :

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

# la app

# Una sola app, dos partes

SITIO PÚBLICO · /

## Para los jugadores

— Sin login: los identifica una cookie firmada con su token.

— Sus controladores declaran allow_unauthenticated_access.

— Jugar, Mi racha, Tabla de posiciones.

BACKEND · /admin

## Para admins y editores

— Login con email y contraseña (bin/rails g authentication).

— Controladores en app/controllers/admin/ que heredan de Admin::BaseController.

— Los permisos de cada rol se controlan ahí.

```
# config/routes.rb
namespace :admin do
  resources :cryptics
end
```

# estructura

# El esqueleto del proyecto

```
rubipticos/
├── app/
│   ├── controllers/
│   │   ├── admin/   # backend, con login
│   │   ├── games_controller.rb   # sitio público
│   │   └── sessions_controller.rb   # g authentication
│   ├── models/
│   │   ├── cryptic.rb  player.rb  game.rb
│   │   └── user.rb   # g authentication
│   ├── services/   # lógica de negocio
│   ├── views/
│   └── javascript/controllers/
├── config/
│   ├── routes.rb
│   └── initializers/inflections.rb   # plurales
├── db/
│   ├── migrate/
│   ├── schema.rb
│   └── seeds.rb
├── test/
└── Gemfile
```

- Lo crea Rails (rails new o un generador)
- Lo hacen ustedes

CARPETAS CLAVE

**app/services**
Clases Ruby comunes: puntaje, racha y sincronización.

**config/routes.rb**
Las URLs del sitio y del admin.

**db/seeds.rb**
Lo que carga bin/rails db:setup.

# lineamientos

# Lineamientos generales

- **Lógica fuera de los controladores**
  En modelos y en servicios (app/services).
- **Validar dos veces**
  En el modelo y en la base: null: false, índices únicos y claves foráneas.
- **Fechas de Argentina**
  config.time_zone = "America/Argentina/Buenos_Aires" y Date.current.
- **Un sistema que arranca solo**
  bin/rails db:setup deja todo listo: catálogo sincronizado y un admin creado.
- **Convenciones de Rails (Convention over Configuration)**
  Nombres, carpetas y rutas REST como espera el framework.

OJO CON

— Rails pluraliza en inglés: Jugador queda «jugadors». Definir el plural en config/initializers/inflections.rb.

— Guarden el token del jugador con cookies.signed. Si alguien edita la cookie, Rails la descarta.



# scaffold

# Scaffolding

**EN LA CONSOLA**

```
$ bin/rails g model Cryptic statement:text answer cryptic_type difficulty
$ bin/rails g scaffold_controller admin/cryptic --model-name=Cryptic
# Admin::CrypticsController, con sus vistas en app/views/admin/cripticos/
```

**SÍ PARA EL ADMIN**

- El ABM de cripticos, usuarios y puntos por tipo.
- Arranca con index, show, new, edit y destroy listos.
- Revisar lo generado: sacar las acciones que no usan y ajustar los parámetros permitidos.

**NO PARA EL JUEGO**

- Una partida no es un ABM: tiene su propio flujo (jugar, responder, pedir pista, rendirse).
- Esos controladores y vistas conviene escribirlos a mano.

bin/rails g scaffold Cryptic ... genera todo junto: modelo, migración, controlador, vistas, rutas y tests.

# vistas

# Las vistas

- **Layouts**
  application.html.erb envuelve todas las páginas. El backend usa el suyo: admin.html.erb.
- **Parciales**
  Pedazos reutilizables, como _form.html.erb, que se incluyen con render.
- **form_with**
  Formularios ligados a un modelo, con protección CSRF y errores de validación.
- **Mensajes flash**
  notice y alert: avisos que se muestran una sola vez, desde el layout.
- **Turbo Frames**
  Actualizan una parte de la página sin recargarla, como la zona de las pistas.

```ruby
# app/controllers/admin/base_controller.rb
class Admin::BaseController
  < ApplicationController
  layout "admin"
end
```

```erb
# app/views/admin/cryptics/new.html.erb
<h1>Nuevo críptico</h1>
<%= render "form",
      cryptic: @cryptic %>
```

Con Bootstrap, alcanza con usar sus clases en las vistas: `class="btn btn-primary"`.

# stimulus

# Stimulus para JavaScript

Pequeños controladores de JavaScript que se enganchan al HTML con atributos data-. Sirven para lo que pasa solo en el navegador (como el timer)

- **data-controller**
  Conecta un elemento con su controlador.
- **data-…-target**
  Marca los elementos que el JavaScript va a tocar.
- **data-…-value**
  Le pasa datos desde la vista, como la hora de inicio.
- **data-action**
  Conecta un evento con un método: click->pistas#pedir.

```erb
# app/views/partidas/show.html.erb
<div data-controller="cronometro"
     data-cronometro-inicio-value=
       "<%= @partida.iniciada_at.to_i %>">
  <span data-cronometro-target="reloj">0 s</span>
</div>
```

```javascript
# app/javascript/controllers/cronometro_controller.js
import { Controller } from "@hotwired/stimulus"
export default class extends Controller {
  static targets = ["reloj"]
  static values = { inicio: Number }
  connect() {
    this.timer = setInterval(() => this.tick(), 1000)
  }
  disconnect() { clearInterval(this.timer) }
  tick() {
    const seg = Math.floor(Date.now() / 1000) - this.inicioValue
    this.relojTarget.textContent = `${seg} s`
  }
}
```

Se crea con bin/rails g stimulus cronometro. El reloj es solo visual: el tiempo oficial lo mide el servidor.

# comandos

# Comandos utiles

## GENERAR Y MIGRAR

- **`bin/rails g model Jugador nombre`**
  Modelo y migración.
- **`bin/rails g controller Partidas`**
  Controlador y vistas.
- **`bin/rails g stimulus cronometro`**
  Un controlador de Stimulus.
- **`bin/rails db:migrate`**
  Aplica las migraciones pendientes.
- **`bin/rails db:rollback`**
  Deshace la última migración, para corregirla.
- **`bin/rails db:seed`**
  Carga los datos de db/seeds.rb.

## LEVANTAR Y REVISAR

- **`bin/setup`**
  Instala gemas, prepara la base y levanta el servidor.
- **`bin/rails db:setup`**
  Crea la base, carga el esquema y los seeds.
- **`bin/dev, o rails s`**
  Levanta el servidor en localhost:3000.
- **`bin/rails console`**
  Probar modelos y consultas a mano.
- **`bin/rails routes`**
  Ver todas las rutas de la aplicación, pueden filtrar con | grep
- **`bin/rails test`**
  Correr los tests.

g es la abreviatura de generate. Para deshacer un generador: bin/rails destroy model Jugador.

# git

# Trabajo en equipo con Git

EN LA CONSOLA

```
$ git checkout -b racha #crear branch
$ git status #para ver que cambios hay
$ git add .
$ git commit -m "Calcula la racha"
$ git push -u origin racha
# pull request en GitHub y merge a main
$ git checkout main
$ git pull #siempre!!
```

- **Una rama por tarea**
  Nadie trabaja directo sobre main.
- **Pull requests**
  Otrx integrante revisa antes de mergear.
- **Commits chicos y claros**
  El mensaje dice qué hace.
- **Commits de todos**
  Quien no tiene commits no puede ser evaluado.